

Automation Anywhere Testbench

An automated test suite for Automation Anywhere Community Edition — what it does, why it exists, how to run it, and the four hidden problems that were blocking every test in it.

15 PASSED	1 SKIPPED	0 FAILED	2 USE CASES
---------------------	---------------------	--------------------	-----------------------

Framework — Playwright (JavaScript), Page Object Model

Repository — github.com/arpittiwari214/aa-automation-tests

Results from a real run against the live Control Room

What's in this guide

01 What this project is

The assignment, in plain words and in technical terms

02 Aim and objectives

Why automate a test at all, and what Page Object Model means

03 How it is built

Folder by folder, and how a test actually executes

04 Running it

Every command, for this laptop and a fresh one

05 The sixteen tests

Each test and what it proves

06 What was actually wrong

Four invisible problems and the evidence for each

07 The skipped test

Why one test is honestly skipped rather than faked green

08 API reference

Every endpoint, captured from real traffic

09 Troubleshooting

Every error encountered, and its fix

10 Assignment audit

The brief, line by line

11 Glossary

Jargon, translated

What this project is

A robot that tests a robot-builder — written twice over, once through a browser and once straight to the server.

IN PLAIN WORDS

Automation Anywhere is a website where companies build small programs that do repetitive office work. The industry calls this RPA — robotic process automation.

The assignment was to check that this website works properly, but without a human clicking through it. So we wrote code that opens a browser, logs in, drags things around, fills in fields, clicks Save, and then checks that each of those things actually happened. If something breaks, the code says exactly which step broke. Then we did the same job a second way, by talking to the server directly with no browser at all.

TECHNICALLY

A Playwright test suite exercising Automation Anywhere Community Edition across two surfaces: a browser-driven UI suite following the Page Object Model, and an HTTP suite driving the Control Room REST API through an API client object. Both assert on observable outcomes rather than incidental UI state — the save test asserts on the actual HTTP response returned by the save call, not merely that a toast appeared.

The two halves

SUITE	WHAT IT DOES	RUNTIME
Use Case 1 Browser · 8 tests	Pretends to be a person. Logs in, creates a form, drags a Textbox and a Select File element onto the canvas, verifies each element's properties panel, types text, saves, and confirms it persisted after a reload.	~1 min
Use Case 2 API · 8 tests	Skips the browser entirely. Authenticates, resolves a workspace folder, creates a Form file, saves its content and dependencies, creates a Process with a three-node workflow, and links the form to it.	~5 sec

Aim and objectives

Why bother automating a test?

IN PLAIN WORDS

Checking a website by hand takes twenty minutes and you get bored by the fourth click. Do it every day for a month and you will start skipping steps without noticing.

A written test never gets bored. It performs the same twenty minutes in one minute, does every step in exactly the same order every time, and tells you precisely which step broke — with a screenshot, a video, and a replay of the page at the moment it failed.

TECHNICALLY

The objective is a regression suite: a deterministic, repeatable specification of expected behaviour that runs on every change and localises a failure to a single step with a trace attached. Testing both the UI and the API matters because they fail differently — a UI test catches a broken button the API knows nothing about, while an API test catches a broken contract long before any UI reveals it, and runs an order of magnitude faster.

The design rule the brief required

PAGE OBJECT MODEL, IN PLAIN WORDS

Instead of scattering "the Save button is over there" across twenty different tests, each screen gets one file that knows where its buttons are. When the website moves a button, you fix one line in one file — not twenty tests.

PAGE OBJECT MODEL, TECHNICALLY

Locators and interactions are encapsulated per screen in `pages/`. Specs express intent and contain no selectors. The API suite mirrors this with a client object in `api/`, so endpoint shapes live in one place rather than being repeated across tests.

OBJECTIVES, STATED PLAINLY

1. Automate the full form-building flow through the browser, with real assertions at each step.
2. Automate the same domain through the API, asserting status codes and response shapes.
3. Keep credentials out of the code entirely.
4. Make failures diagnosable by anyone, not just the author.

How it is built

Four folders, each with one job.

```

aa-automation-tests/
├─ pages/
│   ├─ LoginPage.js           Page Object Model – Use Case 1
│   │                               control room sign-in
│   ├─ AutomationPage.js     Automation menu, Create > Form
│   └─ FormBuilderPage.js    the form designer canvas
├─ api/                      API layer – Use Case 2
│   ├─ endpoints.js          every route, in one place
│   ├─ AaApiClient.js        auth + repository calls
│   └─ payloads.js           form and workflow request bodies
├─ tests/
│   ├─ useCase1.spec.js       8 UI tests, one per assignment step
│   └─ useCase2.spec.js       8 API tests, one per assignment step
├─ scripts/                  investigation tools
│   ├─ capture-auth.js        records the real login call
│   ├─ capture-repo-calls.js  records create/content/dependency calls
│   ├─ debug-panel.js         dumps properties-panel fields
│   ├─ debug-preview.js       proves the upload control is inert
│   ├─ debug-devices.js       checks whether a bot can run at all
│   └─ probe-create.js        prints raw API error responses
├─ test-data/
├─ docs/
├─ playwright.config.js
└─ .env.example

```

What happens when a test runs

#	STEP
1	Playwright reads <code>playwright.config.js</code> and starts the matching project — <code>usecase1-ui</code> or <code>usecase2-api</code> .
2	Credentials are loaded from <code>.env</code> , which is never committed.
3	For UI tests, a Chromium browser launches headless and the spec calls page objects — never raw selectors.
4	Each page object routes canvas locators through the designer iframe.
5	Assertions run. Failures capture a screenshot, a video, and a full trace.
6	Results are written to an HTML report opened with <code>npm run report</code> .

CREDENTIALS

Username and password live in a `.env` file on your own machine, which git is configured to ignore. They are never written into the code and never reach GitHub. This was verified before publishing — no credential appears anywhere in the repository's tracked content.

Running it

On a machine that is already set up

GO TO THE PROJECT – NOTE IT IS A SUBFOLDER

```
PS> cd "D:\Ashutosh\Automation Anywhere\aa-automation-tests"
```

RUN EVERYTHING – ABOUT ONE MINUTE

```
PS> npm test
```

ONLY THE BROWSER TESTS

```
PS> npm run test:usecase1
```

ONLY THE API TESTS – ABOUT FIVE SECONDS

```
PS> npm run test:usecase2
```

OPEN THE REPORT – SCREENSHOTS, VIDEO, REPLAY

```
PS> npm run report
```

VISUAL MODE – WATCH EACH STEP EXECUTE

```
PS> npx playwright test --ui
```

THE REPORT SHOWS ONLY THE LAST RUN

Running just Use Case 1 and then opening the report shows 8 tests, not 16. Run `npm test` first for the complete picture.

On a fresh machine

1 – GET THE CODE

```
PS> git clone https://github.com/arpittiwari214/aa-automation-tests.git
PS> cd aa-automation-tests
```

2 – INSTALL LIBRARIES AND THE BROWSER (LET THIS FINISH COMPLETELY)

```
PS> npm install
PS> npx playwright install chromium
```

3 – CREATE THE CREDENTIALS FILE

```
PS> copy .env.example .env
PS> notepad .env
```

```
PS> npm test
```

FILLING IN .ENV

Set `AA_USERNAME` and `AA_PASSWORD`. Leave `AA_FORM_URL` **empty** — empty makes the suite build its own form from scratch, which is what the assignment asks for. Filling it in makes the tests skip the creation steps.

ONE MACHINE AT A TIME

Community Edition allows one active session per account. If two machines run the suite simultaneously they evict each other's login, and the tests fail in ways that look like bugs but are not.

The investigation tools

SCRIPT	WHAT IT ANSWERS
<code>capture-auth.js</code>	What is the real login endpoint, and what does it return?
<code>capture-repo-calls.js</code>	Which endpoints does the site use to create and save a form?
<code>debug-panel.js</code>	What are the real attributes of every field in the properties panel?
<code>debug-preview.js</code>	Is the file-upload control operable anywhere?
<code>debug-devices.js</code>	Is any device registered that could run a bot?
<code>probe-create.js</code>	What does the server actually say when a request is rejected?

The sixteen tests

USE CASE 1 – BROWSER • 8 TESTS

Step 1 – logs in to the control room with valid credentials	PASS
Steps 2–4 – creates a new Form from the Automation menu	PASS
Step 5 – drags the Textbox and Select File elements onto the canvas	PASS
Step 6 – opens each element and verifies its properties panel	PASS
Step 7a – accepts and retains text in the textbox	PASS
Step 7b – uploads a document and shows the filename as confirmation	SKIPPED
Step 8 – saves the form and verifies the backend accepted it	PASS
Step 8b – the saved form definition survives a reload	PASS

USE CASE 2 – API • 8 TESTS

Step 1 – authenticates and captures a valid auth token	PASS
Step 2 – retrieves the private workspace folder id	PASS
Step 3 – creates a Form file and gets back a valid id	PASS
Step 4 – saves form content with TextBox, TextArea and Number fields	PASS
Step 5 – saves the form file dependencies	PASS
Step 6 – creates a Process file and gets back a valid id	PASS
Step 7 – saves the 3-node workflow referencing the form	PASS
Step 8 – links the form as a dependency of the process	PASS

A NOTE ON STEP 7

The workflow test does not merely check a status code. It asserts the workflow contains exactly three nodes in the order `InitialStep → FormStep → exit`, and that both `InitialStep` and `FormStep` reference the form file created earlier — precisely what the brief specifies.

What was actually wrong

Four problems. Every one looked like a broken test and turned out to be the application behaving in a way nobody could see.

01 Every button label starts with an invisible character

IN PLAIN WORDS

The buttons look like they say "Select File". They don't. Each name secretly begins with a hidden icon character that a font draws as a small picture — invisible on screen, but very real to code.

TECHNICALLY

Palette and menu labels are prefixed with `U+F1C1`, a Private Use Area glyph. Any pattern anchored with `^` can never match, because the first character is never the letter you can see.

```
X /^Select File/      never matches
✓ /Select File$/      matches
```

Matching the end of the name instead of the start fixed the form-creation and drag-and-drop steps together. It is the single most transferable lesson in this project.

02 The form designer is a page inside the page

IN PLAIN WORDS

The area where you drag elements around is not really part of the page — it is a separate mini-page embedded inside it. Code looking at the outer page simply cannot see anything in there.

TECHNICALLY

The designer runs in an iframe, so a top-level `page.locator('.file-upload-content')` never matches. There is a second trap: the site also embeds a third-party help widget in its own iframe, so selecting "the first iframe" worked only by accident of DOM order.

```
page.locator('iframe[src*="modules/attended"]').contentFrame()
```

Selecting the designer by its address removes the ordering dependency entirely.

03 Preview is a photograph, not a form

IN PLAIN WORDS

This explains a mystery that consumed hours. Clicking "browse" on the file box does nothing — not for the test, and not for a real person either. It is not a broken browser, not an extension, and not a bad click.

TECHNICALLY

The builder canvas mounts no `<input type="file">` and fires no filechooser event. Preview does not render a live form at all: it displays a static JPEG inside a modal dialog, and that image intercepts every pointer event.

```

from <div role="dialog" aria-modal="true">
  subtree intercepts pointer events
```

The upload control is decoration in the editor. It only becomes real when a bot runs the form.

04 The API was one character away, then one folder away

IN PLAIN WORDS

The entire API half was stuck on two small things: the login address was version 2 rather than version 1, and files cannot be created in the top-level folder — only inside a folder within it.

TECHNICALLY

`/v1/authentication` returns 404; Community Edition serves `/v2/authentication`. Creating in the workspace root returns `400 repository.exception.root.folder`.

WHY GUESSING WAS WORSE THAN USELESS

The gateway answers **401** for any address that does not exist, including deliberately nonsense ones. A 401 therefore proves nothing about whether a route is real, so endpoints were recorded from the site's own network traffic instead — which is exactly what the assignment meant by "identify endpoints using the Network tab".

The skipped test

Fifteen tests pass. One — Step 7b, uploads a document — is marked skipped, with its reason printed in the report.

WHY NOT JUST MAKE IT GREEN?

It would have been easy. Assert something weak that happens to be true, and the report shows sixteen ticks. But a green tick there would claim an upload was verified when no upload happened. That is worse than having no test at all, because it hides the gap instead of showing it.

TECHNICALLY

A hollow assertion satisfies the runner while misrepresenting coverage. An explicit `test.skip()` carrying its cause keeps the limitation legible in the report rather than laundering it into a pass.

Can it be closed?

Only by running the form for real, which requires the Automation Anywhere **Bot Agent** — a separate Windows application — installed and registered as a device. This account has zero devices registered, so no form can execute at all:

```
PS> node scripts/debug-devices.js
auth: 200
POST /v2/userdevices → 200

devices registered: 0
```

And even with the agent installed, an attended form opens in the Bot Agent's own desktop window rather than a browser page — so Playwright still could not drive it. The result would be a manual demonstration, not a test. That is why it remains a documented skip.

API reference

Every endpoint below was captured from the Control Room's own traffic, not guessed.

PURPOSE	ROUTE AND BODY
Authenticate	POST /v2/authentication body: { username, password, captcha } → { token, user, permissions, tenantUuid, ttlSeconds }
List workspace	POST /v2/repository/workspaces/private/files/list body: { page, sort }
List a folder	POST /v2/repository/folders/{folderId}/list
Create a file	POST /v2/repository/files body: { name, parentFolderId, description, contentType }
Save content	PUT /v2/repository/files/{id}/content?hasErrors=false body: { form }
Save dependencies	PUT /v2/repository/files/{id}/dependencies body: { childFileIds }
List devices	POST /v2/userdevices

THREE DETAILS THAT COST REAL TIME

The destination folder travels in the *body* as `parentFolderId`, not in the URL path. Both save calls are **PUT**, not **POST**. And the authentication header is `X-Authorization`, not a bearer token.

Content types

FILE TYPE	CONTENTTYPE
Form	application/vnd.aa.form
Process / workflow	application/vnd.aa.workflow

Troubleshooting

Every error actually encountered while building this, and its fix.

"Could not read package.json"

You are one folder too high. The project lives in a subfolder.

```
PS> cd "D:\Ashutosh\Automation Anywhere\aa-automation-tests"
```

Your prompt should end in `\aa-automation-tests>` before running npm. A new terminal starts in the wrong place again.

"Executable doesn't exist at ...chrome.exe"

Run it again — this is intermittent. The file is genuinely present; Playwright's existence check fails while Windows Defender holds a lock on the 220 MB binary during a scan. If it persists, repair the install:

```
PS> npx playwright install chromium
```

Excluding `C:\Users\PC\AppData\Local\ms-playwright` from Defender's real-time scanning stops it permanently — but that is a security setting, so decide for yourself whether you want it.

Login fails and the screenshot is blank

The site failed to load. This occurred once after logging in roughly ten times in quick succession during debugging, and cleared on its own. Wait a minute and run again. Also confirm the other machine is not running the suite at the same time.

"spawn UNKNOWN" when using HEADED=1

Expected on this laptop — headed Chrome fails with a Windows side-by-side configuration error, unrelated to the tests. Use visual mode instead:

```
PS> npx playwright test --ui
```

Tests fail on "Missing credentials"

`.env` still holds placeholder text, or does not exist yet.

```
PS> copy .env.example .env
PS> notepad .env
```

The report shows 8 tests instead of 16

The report shows whatever ran last. Run the whole suite, then open it.

```
PS> npm test
PS> npm run report
```

Assignment audit

Checked against the assignment document, item by item.

GENERAL INSTRUCTIONS

Framework: Playwright or Cypress	DONE
Page Object Model for UI automation	DONE
Assert UI element visibility and interaction	DONE
Assert functional flow and expected behaviours	DONE
Assert API responses and functional correctness	DONE

USE CASE 1 – EXPECTATIONS

Build and automate a form with Text Box and File Upload	DONE
Assert UI element visibility and functionality	DONE
Assert file upload status and confirmation	BLOCKED
Assert form submission behaviour and backend response	DONE

USE CASE 2 – EXPECTATIONS

Identify required endpoints from the Network tab	DONE
Assert status codes 200 / 201 at every step	DONE
Form creation response contains a valid id	DONE
Form content save response is successful	DONE
Form dependencies save response is successful	DONE
Process creation response contains a valid id	DONE
Process content save response is successful	DONE
Process dependencies save response is successful	DONE

SUBMISSION GUIDELINES

Code submitted in a GitHub repository	DONE
README with setup and execution instructions	DONE
README lists framework and tools used	DONE
README covers environment and configuration notes	DONE

THE SINGLE BLOCKED ITEM

File upload status and confirmation is the only requirement not met, and it is blocked by the application rather than by the tests. Twenty of twenty-one line items are implemented and passing. See Chapter 07.

Glossary

RPA — Robotic Process Automation

Software that performs repetitive computer work a person would otherwise do by hand.

Playwright

A tool that drives a real browser from code — clicking, typing and checking, exactly as a person would.

Page Object Model (POM)

A way of organising test code so each screen has one file describing where its controls are, keeping tests readable and fixes local.

Assertion

The line that decides pass or fail. "I expect this box to now contain the text I typed."

Locator

The instruction that finds a specific element on screen — by its label, role, or a class name in the page's code.

Selector strictness

Playwright refuses to act if a locator matches more than one element, rather than guessing. Several fixes here came from resolving those collisions.

iframe

A complete web page embedded inside another one. Code must step into it deliberately; it cannot see in from outside.

API

The server's own interface, without a browser. Faster and more stable to test, but it cannot catch a visually broken page.

Endpoint

One specific address on the server that performs one job, such as "log this user in" or "create a file".

Token

A temporary pass issued after logging in, attached to every later request to prove who you are.

Headless

Running a browser with no visible window. Faster, and the default here because visible Chrome is broken on this machine.

Trace

A recording of everything the browser did during a test, replayable step by step with a snapshot of the page at each moment.

Bot Agent

Automation Anywhere's desktop application that actually runs automations. Without it registered as a device, no form or bot can execute.